

FitCheck

Your Personal BMI, BMR & Calorie Companion

Project Report

Student Name	Shaurya vashishtha
Registration No.	26BCE11224
Course	Introduction to problem solving and programming -CSE-1021
Faculty	Abha sharma
Language / Tools	Python 3, Git, GitHub
GitHub Repository	https://github.com/shaurya26bce11224-svg/Fitcheck_project.git
Submission Date	29/09/2026

Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-Functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References

1. Introduction

Everybody hears about BMI, BMR and body fat at some point, usually from a gym trainer, a fitness app or a doctor's visit. But if you ask a friend how those numbers are actually calculated, most people have no idea. I noticed that even when I wanted to check my own numbers, I ended up jumping between different websites, each with its own layout, its own ads and its own way of asking for my details.

That everyday annoyance is where FitCheck came from. It is a small console application written in Python that brings four commonly used health calculations into one menu: BMI, BMR, TDEE and body fat percentage. It works completely offline, does not save any data, and explains its results in plain sentences instead of just throwing out a number.

Beyond being useful, the project was a chance to apply the core ideas from the course in a real setting: functions, conditional logic, loops, user input and output, and turning a mathematical formula into working code.

Objectives

- Bring BMI, BMR, TDEE and body fat calculations together in a single easy-to-use tool.
- Apply well-known, published formulas correctly and consistently.
- Give results in plain English (for example, "healthy weight") rather than only raw numbers.
- Keep the program simple, readable and runnable on any computer with Python installed.
- Keep user data private by working fully offline and storing nothing.

2. Problem Statement

People who want to understand their health numbers have to search for separate online calculators. These are scattered, often cluttered with ads, and sometimes ask for personal information that gets sent to a server. Many people also do not understand what the numbers mean once they get them.

There is a need for a simple, offline tool that collects the most useful calculations in one place, applies the correct formulas, and explains the outcome clearly.

Scope

In scope: BMI with category, BMR for males and females, TDEE with five activity levels, body fat estimation for adult men, adult women, boys and girls, all through a looping console menu.

Out of scope: medical diagnosis, diet or workout plans, user accounts, data storage, internet connectivity and a graphical interface.

Target Users

Students, beginners at the gym, and anyone curious about their body metrics who does not want to memorise formulas. No technical or medical knowledge is required.

3. Functional Requirements

FitCheck is made up of four major functional modules plus the menu that ties them together (the course guideline asks for at least three).

ID	Module	Requirement	Input	Output
FR1	BMI	Calculate BMI = weight / height ² and classify it as underweight (up to 18.5), healthy (up to 24.9), overweight (up to 29.9) or obese.	Height (m), weight (kg)	BMI value and category
FR2	BMR	Calculate basal metabolic rate with the Mifflin-St Jeor equation, using the male or female version.	Gender, height (cm), weight (kg), age	BMR in kcal/day
FR3	TDEE	Calculate BMR, then multiply by the activity factor: 1.2, 1.375, 1.55, 1.725 or 1.9.	Gender, height, weight, age, activity level	TDEE in kcal/day
FR4	Body fat	Estimate body fat % from BMI and age using the Deurenberg equations for adult male, adult female, boy or girl.	Group, height (m), weight (kg), age	Body fat %
FR5	Menu	Show a numbered menu, keep looping until 0 is chosen, and warn on unknown choices.	Menu number	Chosen module or exit

User workflow in short

- Start the program and see the main menu.
- Type the number of the calculation you want.
- Answer the prompts one by one (height, weight, age and so on).
- Read the result, then the menu appears again so you can run another calculation or exit with 0.

4. Non-Functional Requirements

These describe how well FitCheck should behave, not just what it does. I have also noted where the current version stands, because it is more useful to be honest about it.

Requirement	What it means for FitCheck	Current status
Usability	Clear numbered menu, plain-language prompts and results that a non-technical person can follow.	Met. Menu and messages are simple; unit prompts could be more consistent.
Performance	Every calculation should return instantly, since it is only basic arithmetic.	Met. Response is immediate.
Privacy / Security	No personal data should be stored, logged or transmitted.	Met. Everything stays in memory and disappears when the program closes.
Portability	Should run on Windows, macOS and Linux with only standard Python 3.	Met. No external libraries are used.

Requirement	What it means for FitCheck	Current status
Maintainability	Formulas should sit in reusable functions so they can be changed in one place.	Partly met. BMI and BMR are functions; TDEE and body fat logic still live in the menu loop.
Reliability / Error handling	Wrong or unexpected input should not crash the program.	Partly met. Unknown menu numbers are handled; non-numeric input still raises an error (see Challenges).
Resource efficiency	Negligible memory and CPU use.	Met.

5. System Architecture

FitCheck follows a simple layered structure. At the top the user talks to the presentation layer (the menu and the input/output prompts). Underneath it, a logic layer decides which calculation to run and turns numbers into categories, such as a BMI value into "healthy weight". At the bottom are the calculation functions and the fixed values used by the formulas, like the activity multipliers.

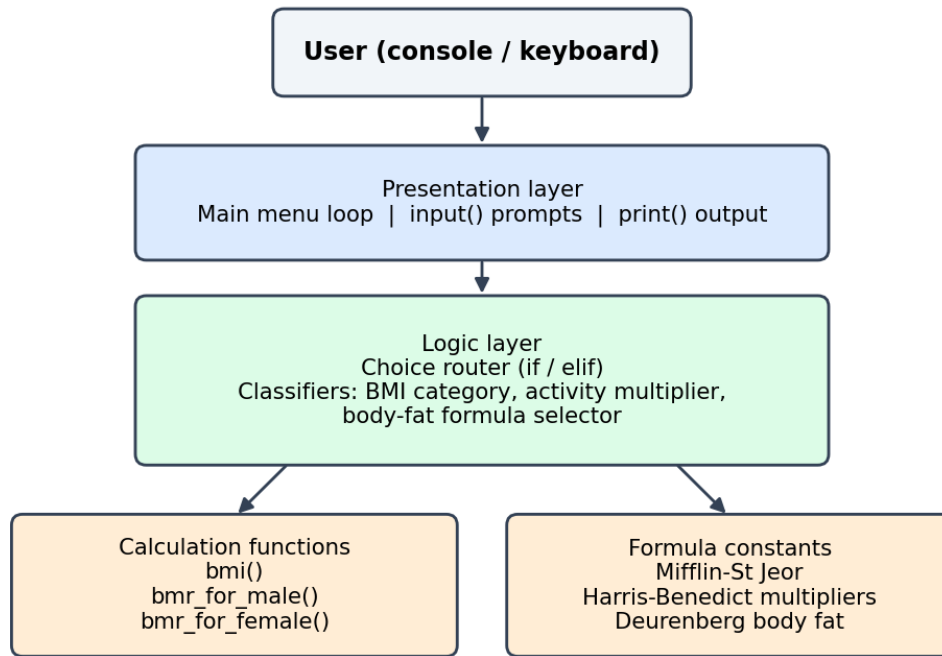


Figure 1: Layered architecture of FitCheck

Since the program stores nothing and needs no network, there is no database or server component. That keeps the design small and easy to understand.

6. Design Diagrams

6.1 Use Case Diagram

There is a single actor, the user. Each calculation is a separate use case, and the TDEE calculation depends on the BMR calculation.

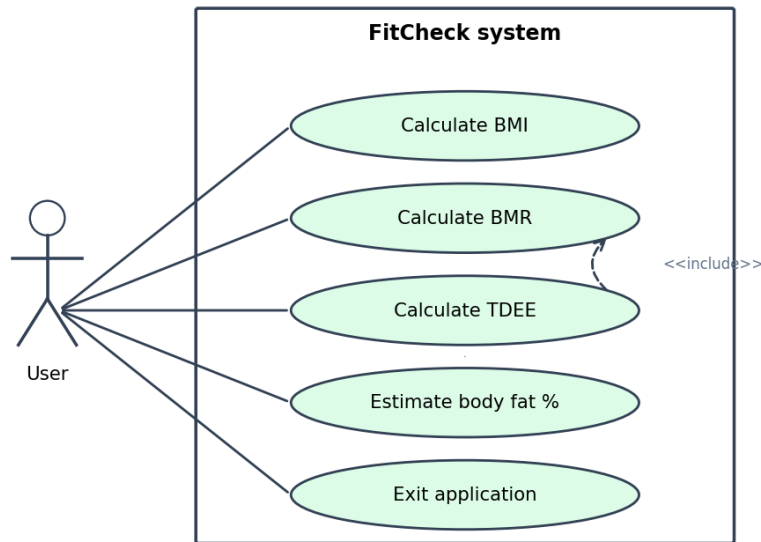


Figure 2: Use case diagram

6.2 Workflow Diagram

The program shows the menu, reads a choice, runs the matching module, prints the result and goes back to the menu. Choosing 0 ends it and any other value prints a warning.

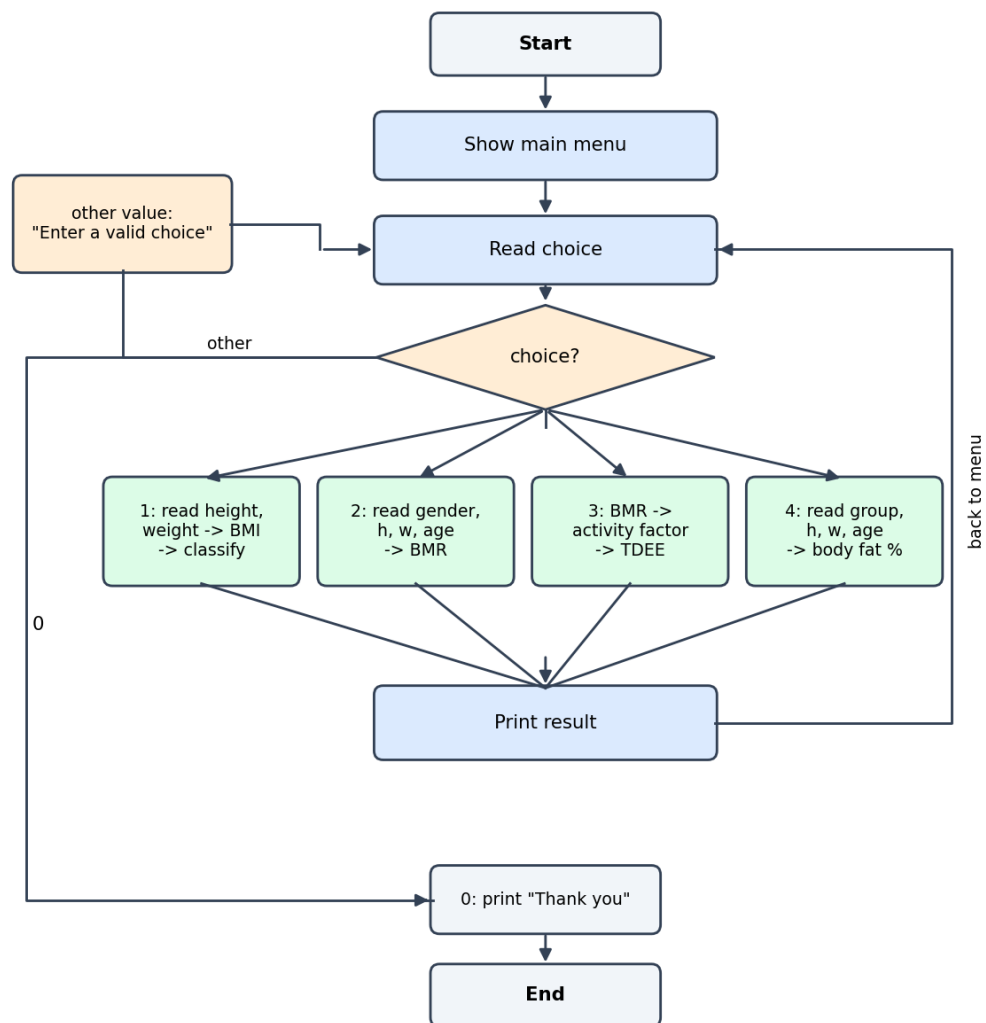


Figure 3: Workflow of the main menu loop

6.3 Sequence Diagram

The diagram below follows one TDEE request from the first keystroke to the printed answer.

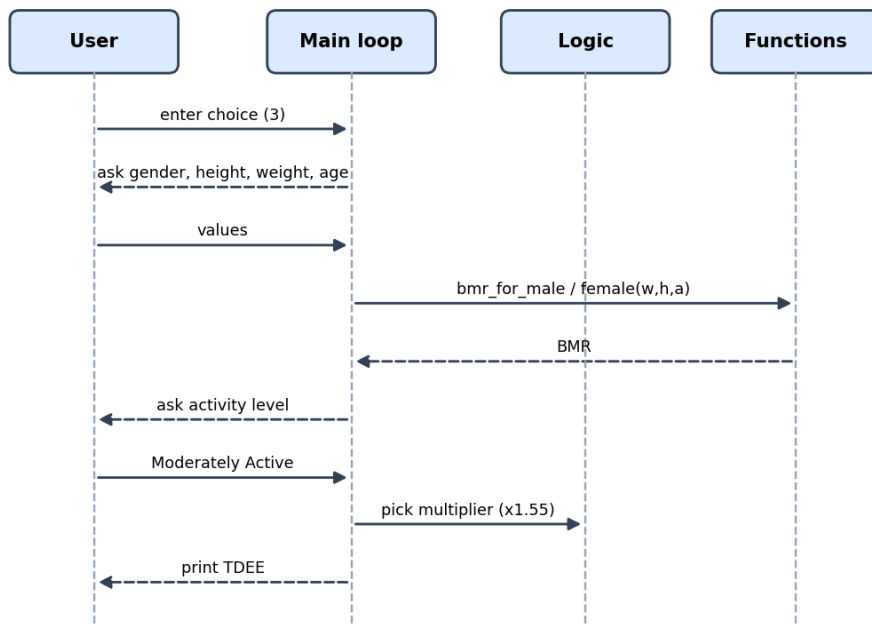


Figure 4: Sequence diagram for a TDEE calculation

6.4 Component Diagram

FitCheck is one Python file, so I have shown it as logical components rather than separate classes. The program has no classes because none were needed for this design.

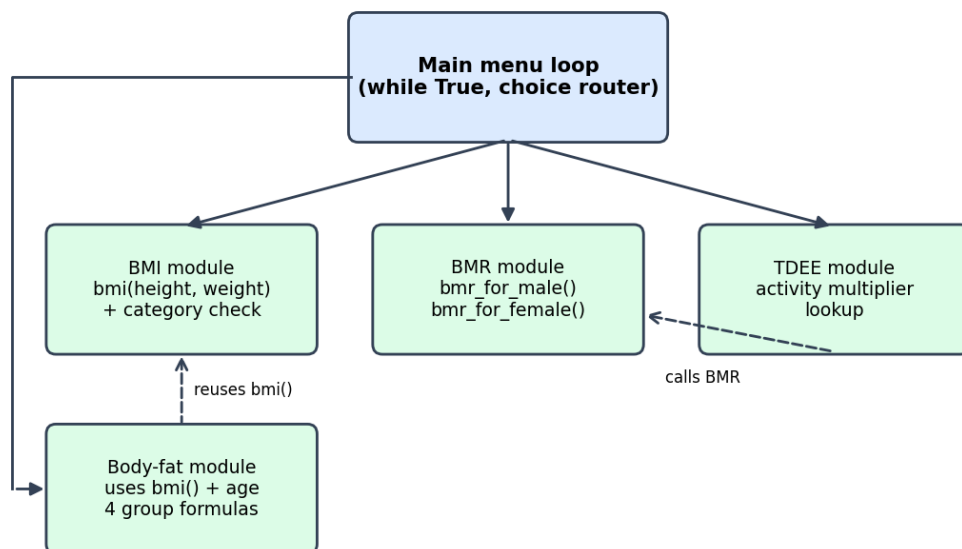


Figure 5: Component diagram

6.5 ER Diagram

Not applicable. FitCheck does not use any database or file storage, so there are no entities to model.

7. Design Decisions & Rationale

Decision	Why I made it
Console application instead of GUI or web app	It keeps the focus on the logic and formulas, which is what the course is about, and it runs anywhere.
Mifflin-St Jeor for BMR	It is widely used and generally regarded as more accurate for modern populations than the older Harris-Benedict equation.
Standard activity multipliers (1.2 to 1.9)	These are the commonly accepted factors for turning BMR into TDEE, so results match what other tools produce.
Deurenberg formula for body fat	It needs only BMI and age, which people actually know, and it has versions for adults and children. Skinfold or tape measurements would make the tool harder to use.
Functions for BMI and BMR	Body fat reuses the BMI function and TDEE reuses the BMR functions, so each formula lives in one place.
`while True` menu loop	It lets people run several calculations without restarting the program.
No data storage	It keeps things private and removes the need for a database.

8. Implementation Details

Everything is in **Nutrifit_Project.py**. The main pieces are described below.

Calculation functions

```
def bmi(a,b):          # a = height (m), b = weight (kg)
    c=b/(a*a)
    return(c)

def bmr_for_male(w,h,a): # Mifflin-St Jeor, male
    bmr=(10*w)+(6.25*h)-(5*a)+5
    return(bmr)

def bmr_for_female(w,h,a): # Mifflin-St Jeor, female
    bmr=(10*w)+(6.25*h)-(5*a)-161
    return(bmr)
```

Formulas used

Metric	Formula
BMI	$\text{weight} / \text{height}^2$
BMR (male)	$10 \times \text{weight} + 6.25 \times \text{height(cm)} - 5 \times \text{age} + 5$
BMR (female)	$10 \times \text{weight} + 6.25 \times \text{height(cm)} - 5 \times \text{age} - 161$
TDEE	$\text{BMR} \times \text{activity factor (1.2, 1.375, 1.55, 1.725, 1.9)}$
Body fat, adult male	$1.2 \times \text{BMI} + 0.23 \times \text{age} - 16.2$
Body fat, adult female	$1.2 \times \text{BMI} + 0.23 \times \text{age} - 5.2$
Body fat, boy	$1.51 \times \text{BMI} + 0.70 \times \text{age} - 2.2$
Body fat, girl	$1.51 \times \text{BMI} + 0.70 \times \text{age} - 1.4$

Menu and control flow

The menu is a `while True` loop that reads the user's choice with `int(input())` and uses an `if / elif` chain to pick the module. BMI values are compared against thresholds to pick a category. TDEE uses a chain of comparisons on the activity level text to pick the multiplier. Option 0 uses `break` to leave the loop.

Concepts applied from the course

- Functions with parameters and return values.
- Conditional statements (`if / elif / else`) for classification and routing.
- Loops (`while`) for a repeating menu.
- Type conversion (`int, float`) for user input.
- Arithmetic operators to implement formulas.
- String comparison to match gender and activity level.

9. Screenshots / Results

These are real outputs copied from the terminal while running the program (text captures instead of image screenshots).

BMI (option 1)

```
Enter a choice:1
Enter height in metre:1.75
Enter weight:70
BMI is 22.857142857142858 kg/m² and the person is healthy weight
```

BMR (option 2)

```
Enter a choice:2
Person is male or female:male
Enter hieght:175
Enter wieght:70
Enter age 25
basal metabolic rate is: 1673.75 kilocalories per day
```

TDEE (option 3)

```
Enter a choice:3
Person is male or female:male
Enter hieght:175
Enter wieght:70
Enter age 25
basal metabolic rate is 1673.75 kilocalories per day
Activity level is Sedentary or Lightly Active or ...:Moderately Active
Total Daily Energy Expenditure of Body is 2594.3125 kilocalories per day
```

Body fat (option 4)

```
Enter a choice:4
Person is Adult male or Adult female or Boy or Girl:Adult male
Enter height: 1.75
Enter weight: 70
Enter age: 25
Body fat is 16.97857142857143 %
```

Invalid menu choice

```
Enter a choice:9
Enter a valid choice
```

10. Testing Approach

Because the program is calculation-heavy, I used **validation testing**: pick sample inputs, work out the answer by hand from the formula, run the program and compare. I ran each module through the terminal, feeding in inputs as if a user were typing them.

Test	Input	Expected	Actual	Result
BMI, healthy	1.75 m, 70 kg	22.86, healthy	22.857..., healthy	Pass
BMR, male	175 cm, 70 kg, 25 y	1673.75	1673.75	Pass
BMR, female	165 cm, 60 kg, 22 y	1360.25	1360.25	Pass
TDEE, moderate	male, 175, 70, 25, Moderately Active	2594.31	2594.3125	Pass

Test	Input	Expected	Actual	Result
Body fat, adult male	1.75 m, 70 kg, 25 y	16.98 %	16.978... %	Pass
Body fat, adult female	1.65 m, 60 kg, 22 y	26.31 %	26.306... %	Pass
Unknown menu number	9	Ask for valid choice	"Enter a valid choice"	Pass
TDEE, Sedentary	male, 175, 70, 25, Sedentary	Show 2008.5	Message printed with no number	Fail
Non-numeric menu input	abc	Friendly message	ValueError crash	Fail

The two failures are listed in the next section as known issues. I would rather report them honestly than hide them, and they clearly show what to fix next.

11. Challenges Faced

- **Mixing units.** The BMI formula needs height in metres, while the Mifflin-St Jeor BMR equation needs centimetres. It took me a while to notice why my BMR came out wrong when I first typed 1.75, and the prompts still need to be clearer about which unit is expected.
- **Matching text exactly.** Gender and activity level are compared as exact strings, so Moderately active will not match Moderately Active. Small typos silently cause the program to skip the calculation.
- **A missing value in one branch.** While testing TDEE, I found that the Sedentary branch prints the sentence but never prints the calculated number. Every other activity level prints it correctly, so it was easy to miss until I tested each one separately.
- **Input validation.** Entering letters where a number is expected crashes the program with a ValueError. Handling this needs try/except, which I plan to add.
- **Repeated code.** The BMR input code appears in both option 2 and option 3, which made me realise the program would be cleaner with more helper functions.

12. Learnings & Key Takeaways

- Turning a formula into code is straightforward, but making it usable for a real person needs more thought than the maths itself.
- Testing every branch separately matters. The Sedentary bug only showed up because I tried each activity level one by one.
- Small design choices, like which units to ask for, change how confusing a program feels.
- Reusing functions (BMI inside body fat, BMR inside TDEE) saves effort and reduces mistakes.
- Reading the assignment guidelines closely early on would have saved time on structure, documentation and testing.

13. Future Enhancements

- Add input validation with try/except and retry loops so the program never crashes on bad input.
- Accept case-insensitive text and use numbered choices for gender and activity level.

- Use one consistent unit system, or let the user choose metric or imperial.
- Fix the Sedentary TDEE output and move TDEE and body fat logic into their own functions.
- Split the code into several modules (for example calculators, menu, validation and tests) with a proper folder structure and a unit test suite.
- Add logging and save a history of results to a file so users can track progress.
- Add calorie targets for weight loss or gain and simple diet suggestions.
- Build a graphical or web interface with charts.

14. References

- Mifflin, M. D., St Jeor, S. T., et al. (1990). A new predictive equation for resting energy expenditure in healthy individuals. *American Journal of Clinical Nutrition*, 51(2), 241-247.
- Deurenberg, P., Weststrate, J. A., & Seidell, J. C. (1991). Body mass index as a measure of body fatness: age- and sex-specific prediction formulas. *British Journal of Nutrition*, 65(2), 105-114.
- World Health Organization. Body mass index (BMI) classification.
- Python Software Foundation. Python 3 documentation. <https://docs.python.org/3/>
- VITyarthi. Build Your Own Project: General Project Instructions & Submission Guidelines.